Comp 551 MiniProject 2: Training and Evaluation Practices

Ryu Quan, Junu Seo, Yantian Yin

February 2026

1 Abstract

In this project, we study training and evaluation practices for logistic regression using the UCI Spambase dataset and the objective is to learn a probabilistic classifier that predicts whether an email is spam. To highlight issues of overfitting and generalization, we intentionally train the model on a small subset of the data (5% of the full dataset) and reserve the remaining portion as a held-out test set.

We implement logistic regression from scratch and optimize it using mini-batch stochastic gradient descent (SGD) with L2 regularization, Going beyond the basic requirement, we also incorporate momentum to improve training stability and convergence speed.

In Task 2, however, we used K-fold cross-validation to tune hyperparameters for a mini-batch SGD logistic regression classifier on the UCI Spambase dataset. We held out the test set once at the start and did all tuning only on the small (5%) standardized training split, and shuffling each epoch for reproducibility. We search the learning rate, batch size and epochs in a grid, reporting the mean ± std validation cross-entropy and precision of the mean validation std ± across the folds, then selected the best settings (with fixed) to support the later bias-variance analysis and the final evaluation of the test.

When doing K-fold cross-validation with L2 regularization, we found that small λ led to overfitting and high variance while large λ led to underfitting and high bias. The optimal λ of 10^{-2} minimized validation cross-entropy loss with a value of 0.3274 and achieved an accuracy of 89.98%. We saw similar results for L1 regularization with the difference being that L1 regularization produced more sparse models as regularization strength increases.

2 Introduction

The dataset contains 57 continuous features: 48 word-frequency features, 6 character-frequency features, 1 `run_length_average`, 1 `run_length_longest`, and 1 `run_length_total`. These features represent word and character frequency statistics. The dataset also includes a binary target label $y \in \{0, 1\}$ indicating whether an email is spam (1) or not spam (0). The modeling approach uses mini-batch stochastic gradient descent (SGD) to optimize the logistic regression model with cross-entropy loss.

We selected optimization and model hyperparameters (learning rate, batch size, epochs, and regularization strength) using K-fold cross-validation, choosing the setting with the best average validation performance across folds.

The project emphasizes understanding optimization behavior, the bias–variance trade-off, and principled model selection through K-fold cross-validation. In addition, we analyze how varying the regularization strength affects generalization performance and visualize the regularization path under L1 penalty using a standard library implementation.

3 Methods

3.1 Preprocessing steps

We split the dataset into 5% training data and 95% testing data. We compute the mean and standard deviation using only the training set, and then standardize both the training and testing sets using these statistics. This avoids data leakage, since we do not use any information from the test set when estimating the scaling parameters.

3.2 SGD logistic regression implementation

We implement logistic regression optimized using mini-batch stochastic gradient descent (SGD). First, we define the sigmoid function, which maps the linear score $w^\top x$ to the interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = w^\top x + b.$$

The predicted probabilities are then used to compute the cross-entropy loss. Taking the gradient of the cross-entropy loss with respect to w and b then we have the dw, db for updating the weights in later steps.

In order to do SGD with batches we divide the dataset into small mini-batches. For each batch, we compute the average gradient

$$\nabla_w = \frac{1}{B} X_{\text{batch}}^\top (\hat{y} - y),$$

and update the parameters once per batch using

$$w \leftarrow w - \eta \nabla_w,$$

where η is the learning rate. This constitutes mini-batch SGD.

3.3 L2 regularization convention

We incorporate L2 regularization by optimizing the regularized objective

$$L_{\text{reg}}(w, b) = L(w, b) + \lambda \|w\|_2^2.$$

The gradient of the regularization term is $2\lambda w$, which is added to the weight gradient.

Going beyond, we incorporate momentum to improve the stability and convergence speed of training. We use a momentum coefficient of $\beta = 0.9$. The velocity term is updated according to

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_w,$$

and the parameters are updated using

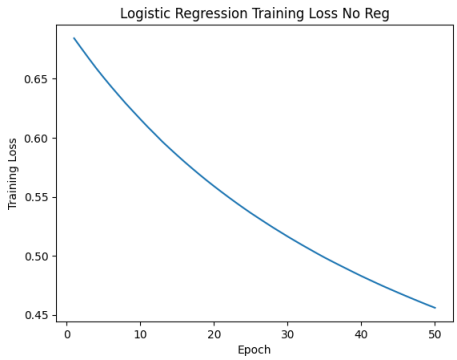
$$w \leftarrow w - \eta v_t.$$

3.4 CV protocol

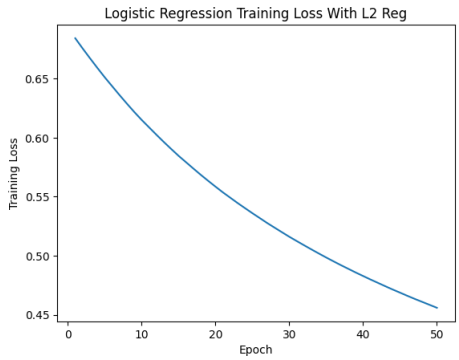
For K-fold cross-validation, we randomly shuffle the training set and split it into 5 equally sized folds. One fold is selected as the validation fold and the rest are combined to form the training fold. The model is then trained on the training fold and evaluated on the validation fold. This process repeats K times, each time selecting a different fold to be the validation fold, and the performance values from all runs are averaged. We repeat this procedure for each hyperparameter setting, including the regularization strength , learning rate , batch size B, and number of epochs. The final hyperparameters are selected on the basis of the best average validation performance across folds.

4 Results

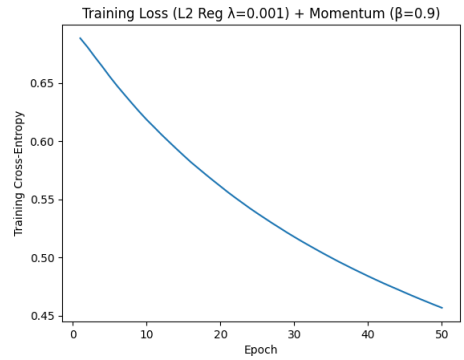
4.1 Task 1: Logistic Regression with SGD



(a) No regularization



(b) L2 ($\lambda = 0.001$)



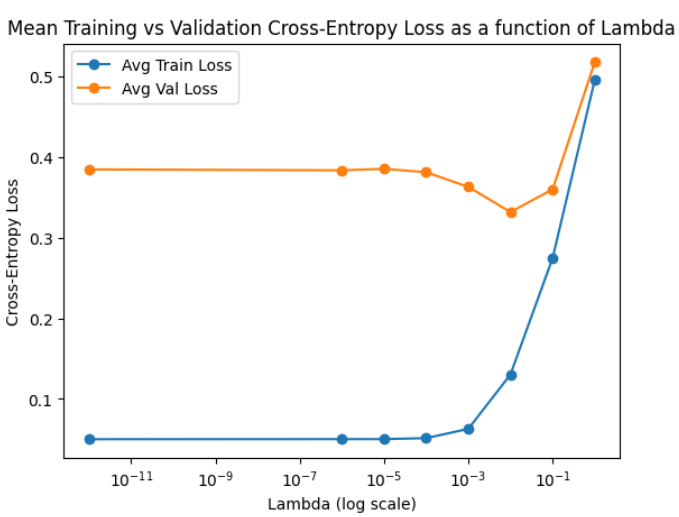
(c) L2 + Momentum (0.9)

4.2 Task 2: Cross-Validation Results and Chosen Hyperparameters

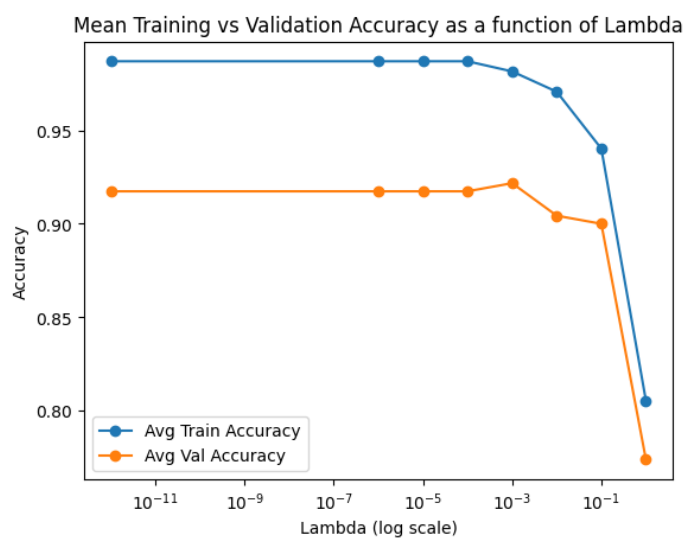
Rank	η	Batch Size	Epochs	CV Log Loss Mean	CV Log Loss Std	CV Acc. Mean	CV Acc. Std
1	0.01	16	100	0.331235	0.133961	0.900000	0.044764
2	0.01	16	50	0.333847	0.093062	0.900000	0.044764
3	0.01	64	100	0.356667	0.063004	0.908696	0.031950
4	0.01	64	50	0.402341	0.041140	0.908696	0.025352
5	0.001	16	100	0.421750	0.035693	0.913043	0.030744
6	0.001	16	50	0.489371	0.021772	0.900000	0.035322
7	0.001	64	100	0.557686	0.012796	0.895652	0.050330
8	0.001	64	50	0.611937	0.007647	0.895652	0.062099

Table 1: CV Results

4.3 Task 3: λ-Sweep Curves and Final Test Results



(a) Cross-Entropy Loss vs λ

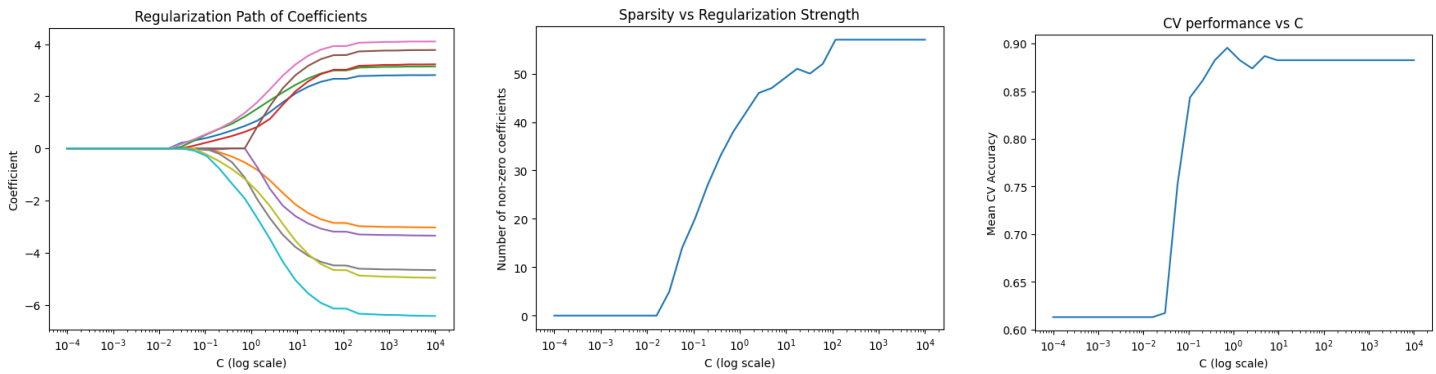


(b) Accuracy vs λ

λ	Cross-Entropy Loss	Accuracy
0.01	0.3274	0.8998

Table 2: Final Test Results using the selected λ

4.4 Task 4: L1 Path, Sparsity, and Performance Discussion



(a) Regularization path (b) Sparsity vs C (c) Cross-validation performance vs C

5 Discussion and Conclusion

5.1 Task 1 Discussion

For task 1 stability, the training loss decreases smoothly across epochs, indicating stable optimization behavior. Interestingly, we observed that using true stochastic gradient descent (batch size $B = 1$) converged faster in terms of epochs compared to mini-batch training.

5.2 Task 2 Discussion

From Table 1, Task 2 reports a 5-fold cross-validation grid search over the learning rate (η), batch size (B), and number of epochs using the standardized 5% training split. Overall, the learning rate had the largest impact on validation cross-entropy: across identical batch sizes and epoch budgets, $\eta = 0.01$ consistently achieved substantially lower mean CV log loss than $\eta = 0.001$, indicating faster and more effective optimization in this small-data regime. With $\eta = 0.01$, the smaller mini-batch (B = 16) produced the best mean log loss (0.331) relative to B = 64, suggesting that the additional gradient noise from smaller batches provided mild regularization and improved generalization. Extending training from 50 to 100 epochs delivered only a marginal gain (0.334 $\rightarrow$ 0.331 for B = 16), implying that most improvement comes from reaching a good solution rather than from substantially longer training. Although some configurations (e.g., $\eta = 0.001$, B = 16, 100 epochs) achieved slightly higher mean accuracy, their log loss was noticeably worse, indicating poorer probability calibration even when 0/1 predictions were occasionally correct. Since logistic regression is trained to minimize cross-entropy, we therefore selected hyperparameters primarily by mean CV log loss. Finally, the relatively large standard deviations in Table 1 are consistent with the high variance of K-fold CV when each fold contains few examples, underscoring the need for cross-validation to support principled hyperparameter selection in this intentionally small-training setting.

5.3 Task 3 Discussion

From Figure 4.3 (a), we observe that when λ is small, the training error is very low while the validation error is high, leaving a large gap between the two curves. This is because the regularization strength is small, so there is less penalty on large weights, leading to the model being too closely fit to the training data, meaning that it captures the noise in the training data rather than the true pattern which explains the strong performance on training data, but very poor performance on unseen data. The large gap between the training error and the validation error when λ is small illustrates the high variance and poor generalization due to overfitting.

As λ increases, the validation loss decreases and reaches a minimum at an intermediate λ, showing improved generalization. For a large λ, both the training and validation errors are large and their curves are close together. The large training error shows the high bias due to underfitting of the model which results in a model that is too simple to capture the pattern in the data and to correctly predict on any data whether it was seen or unseen. The selected λ is 10^{-2} because it was the one that produced the smallest average validation loss, so it is the optimal λ.

5.4 Task 4 Discussion

The regularization path represents how the regularization strength C affects the coefficients. When C is small, there is large regularization which heavily penalizes large weights and causes many coefficients to be 0 in L1 regularization as seen in Figures 4.4 (a) and (b), so the model is simpler and there is more sparsity. As C gets larger, the strength of regularization decreases, so there is less penalty on large weights which causes the coefficients to grow in magnitude and the model becomes more complex with less sparsity. When C is too large, there is overfitting, so the model is fit too closely to the training data.

In L1 regularization, the most important features are the ones that become non-zero the earliest as C increases (regularization decreases). Since L1 regularization can cause some features to become 0, as the regularization strength increases, the model becomes more selective, therefore features that are still non-zero despite the strong regularization have the strongest relationship with the label.

The trade-off between sparsity and performance is that increasing sparsity simplifies the model which may lead to better generalization and thus, better performance when predicting on unseen data, especially if the model was originally overfitted. However, increasing the sparsity too much can eliminate many features, thus making the model too simple and cause underfitting which reduces model performance when predicting on any data.

The difference between L1 and L2 regularization is that as regularization strength increases, L1 regularization can perform feature selection which shrinks some coefficients to zero, resulting is a sparser model, whereas L2 regularization shinks coefficients towards zero but doesn't turn them to zero, so it keeps all the features, but decreases their magnitudes.

5.5 Limitations and Conclusion

A limitation we faced was the small training set size. While a small training set size is good for visualizing the behavior of overfitting, the increased variance may affect the values of the validation errors and as a result, the λ chosen may not be the most optimal value. Possible next steps could be to increase the training set to reduce variance while performing K-fold cross-validation to find a potentially more optimal value of λ.

6 Statement of Contributions

Yantian did task 1, Junu did task 2 and Ryu did task 3 and 4. All team members worked on the write up.

7 Statement on the Use of LLMs and Other Resources

We used a large language model (LLM) to assist with formulating and refining LATEX code and improving the clarity and grammar of the written report. All model implementation, experiments, and analysis were conducted by the group.